

```
root# gdb <binary executable with debug symbols>
GNU gdb (GDB) 7.2ubuntu1
Copyright (C) 2011 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show
warranty" for details. This GDB was configured as "i686-linux-gnu".

For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /sys/kernel/debug/kprobes...done.
(gdb)
(gdb) help
List of classes of commands:
aliases -- Aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without stopping the program
user-defined -- User-defined commands
Type "help" followed by a class name for a list of commands in that class.
Type "help all" for the list of all commands.
Type "help" followed by command name for full documentation.
Type "apropos word" to search for commands related to "word".
Command name abbreviations are allowed if unambiguous.
(gdb)
```

Figure 4: GDB help

```
Id Target Id Frame
5 Thread 20282.20651 0x40030750 in __pthread_cond_timedwait (cond=0x741748,
mutex=<optimized out>, abstime=0x42318d48) at pthread_cond_timedwait.c:159

4 Thread 20282.20650 0x40030750 in __pthread_cond_timedwait (cond=0x73a468,
mutex=<optimized out>, abstime=0x41b18d48) at pthread_cond_timedwait.c:159

3 Thread 20282.20649 0x40030750 in __pthread_cond_timedwait (cond=0x7ab318,
mutex=<optimized out>, abstime=0x41318d48) at pthread_cond_timedwait.c:159

2 Thread 20282.20648 0x40030750 in __pthread_cond_timedwait (cond=0x7d8c70,
mutex=<optimized out>, abstime=0x40b18d48) at pthread_cond_timedwait.c:159

* 1 Thread 20282.20282 Connection::create
```

Figure 5: Threads

For example, on a host PC (normally x86 platform), compile *gdbserver* for the ARM Linux embedded target with *arm-linux-gcc* toolchain.

Once compiled, start the *gdbserver* on the target hardware:

```
root@root# /usr/local/sbin/gdbserver hostip:2345
<<application>>
starting gdbserver :2345
Listening on port 2345
```

Here, 2345 is the TCP port number where the *gdbserver* now passively waits for the connections on the target hardware. On the host side, start the GDB session with the debug version of the executable.

```
Root# arm-linux-gdb <<application executable with debug
symbols>>
```

Then, on the *gdb* prompt, connect to the target using the following command:

```
gdb> target remote target ip:2345
```

Now continue the debugging session with the GDB commands.

## Debugging multithread applications

GDB provides a set of commands to debug multithreaded applications. GDB's thread debugging allows users to observe all the threads that are executing currently on the system, put thread specific breakpoints and display the thread information.

While debugging threads in GDB, only the current thread in execution can be debugged.

### info threads

The above command will display all the current threads in the system as shown in Figure 5. “\*” in Figure 5 indicates the current thread.

To debug other threads, switch between threads and continue with the debugging session.

```
thread <<thread number>>
```

This command is used to set the current thread to the thread that has <<thread number>>. Now, the required thread with <<thread number>> can be debugged.

## Setting breakpoints in threads

When debugging multiple threads, we sometimes need to choose whether to break at a particular line for all threads, or to break at the line number in a file for one particular thread.

If we choose to break for all threads, we just need to set the breakpoint as:

```
break Filename:linenumber
```

If we choose to break for one particular thread, then the breakpoint command will include the thread number as:

```
break Filename:linenumber thread threadnumber
```

The above command will then break at the line number << linenumber >> only when the thread with <<thread number>> hits the breakpoint. 

**By: B B Ramya**

The author is a tech lead at Honeywell Technologies Solutions Lab, Bengaluru and can be reached at ramya3001@gmail.com.